Simulation-Based Evaluation of UAV Navigation Using PPO and Adaptive Control Barrier Functions

B.Tech. Project Report

Dual Degree Programme

Submitted by

Tridibesh Sarkar

Roll Number: 23AE30023

Supervisor

Prof. Priyam Chakraborty

Department of Aerospace Engineering

Indian Institute of Technology Kharagpur

November 2026

Certificate

This is to certify that the project report entitled **"Simulation-Based Evaluation of UAV Navigation Using PPO and Adaptive Control Barrier Functions"**, submitted by **Tridibesh Sarkar** (23AE30023) to the Department of Aerospace Engineering, Indian Institute of Technology Kharagpur, is a record of bona fide work carried out under my supervision and guidance.

Prof. Priyam Chakraborty
Department of Aerospace Engineering
Indian Institute of Technology Kharagpur

Date: _____

Abstract

Autonomous unmanned aerial vehicles must achieve navigation objectives while maintaining adequate separation from moving obstacles. A reinforcement-learning policy can learn useful motion commands, but successful goal reaching alone does not establish obstacle safety. This project implements and evaluates a paper-inspired hierarchical controller that combines proximal policy optimization (PPO), an interacting multiple model (IMM) obstacle predictor, and an adaptive control barrier function (ACBF) filter. The evaluation uses a Crazyflie CF2X model in PyBullet and compares pure PPO, the same PPO with IMM and ACBF, and a classical PID-based reference controller with the same filter.

The implementation uses a 19-dimensional observation, bounded net-acceleration commands, an inner attitude loop, and a staged goal-distance curriculum. A fresh eight-environment training run with a two-million-transition budget stopped at 670,000 transitions after satisfying its repeated-validation condition. The automatically selected and validated checkpoint, saved at 630,000 transitions, is used for the main scenario results. Both this checkpoint and the final checkpoint reached 59 of 60 additional evaluation goals. The remaining target produced a small but persistent position error outside the predefined 0.10 m success radius.

All three controllers reached the shared goal in the linear, accelerating and figure-eight obstacle scenarios. For the selected checkpoint in the figure-eight case, pure PPO entered the 0.50 m safety buffer, attaining a minimum centre distance of 0.464 m. PPO with ACBF maintained 0.770 m and reached the goal in 13.60 s, compared with 13.70 s for pure PPO. The PID-based baseline was faster, reaching the same goal in 7.80 s. These experiments demonstrate effective integration and an observed safety-buffer benefit, while exposing remaining limitations in goal generalization, numerical timing and prediction robustness. The results concern one training seed and a finite set of simulated trials; they do not constitute a general safety guarantee or hardware validation.

Keywords: UAV navigation; proximal policy optimization; interacting multiple model; adaptive control barrier function; safety filtering; PyBullet.

Contents

Certificate i

Abstract ii

Abbreviations and Notation viii

1 Introduction 1

1.1 Motivation . . . 1
1.2 Problem statement . . . 1
1.3 Objectives and scope . . . 2
1.4 Project contribution . . . 2
1.5 Organization of the report . . . 2

2 Background and Related Work 3

2.1 Reinforcement learning and PPO . . . 3
2.2 Safety filtering . . . 3
2.3 Obstacle prediction with multiple models . . . 4
2.4 Simulation platform and classical baseline . . . 4
2.5 Relationship to the source paper . . . 4

3 System Design and Mathematical Formulation 6

3.1 Hierarchical architecture . . . 6
3.2 State, observation and action . . . 6
3.3 Navigation reward and stopping criterion . . . 7
3.4 IMM prediction . . . 8
3.5 Geometry and adaptive barrier conditions . . . 8
3.6 Numerical safety-filter optimization . . . 9
3.7 Meaning and limits of safety . . . 10

4 Implementation and Experimental Method 11

4.1 Software organization . . . 11
4.2 Training configuration . . . 11
4.3 Curriculum and validation . . . 12
4.4 Checkpoint identity and reporting policy . . . 13

4.5 Additional goal evaluation . . . 13
4.6 Moving-obstacle scenarios . . . 14
4.7 Metrics and interpretation . . . 14

5 Results and Discussion **15**
5.1 Training progress and early stopping . . . 15
5.2 Additional goal-reaching tests . . . 16
5.3 Scenario results for the selected model . . . 17
5.3.1 Linear obstacle . . . 18
5.3.2 Accelerating obstacle . . . 19
5.3.3 Figure-eight obstacle . . . 19
5.4 Final-checkpoint comparison . . . 21
5.5 Diagnosis of earlier near-goal failures . . . 21
5.6 Computational and safety limitations . . . 22
5.7 Threats to validity . . . 22
5.8 Discussion . . . 23

6 Conclusions and Future Work **24**
6.1 Conclusions . . . 24
6.2 Future work . . . 25

References **26**

A Reproducibility and Artifacts **27**
A.1 Experiment identity . . . 27
A.2 Recorded dependencies . . . 27
A.3 Evaluation commands . . . 27
A.4 Figure inventory . . . 28
A.5 Interpretation checklist . . . 29

List of Figures

3.1 Implemented hierarchy. The safety layer is used during evaluation; PPO training uses goal-navigation episodes with a distant obstacle. . . 6

5.1 Validation history for the reported eight-environment run. The dashed vertical line marks 507,904 transitions, the end of the earlier same-seed run. Current-stage goals change on curriculum promotion. The repeated end-of-training validation is counted only once in this plot. . . 15
5.2 Observed success on the same additional goal set for the two checkpoints. No out-of-bounds event occurred in these tests. 16
5.3 Selected-checkpoint comparison. In the right panel the dashed line is the 0.50 m safety boundary and the dotted line is the 0.40 m collision threshold. 17
5.4 Goal-error histories for the selected checkpoint. The dotted horizontal line is the 0.10 m position tolerance; arrival additionally requires the speed and hold-time conditions. 18
5.5 Case 1 trajectories for the selected checkpoint. A spatial trajectory plot does not by itself establish time-aligned obstacle clearance; that is measured from the logged states. 18
5.6 Case 2 trajectories for the selected checkpoint. Both RL variants arrive, while the filtered path maintains a larger sampled minimum separation. 19
5.7 Case 3 trajectories for the selected checkpoint. PPO+ACBF and PID+ACBF use the same obstacle predictor and filter but different nominal commands. 20
5.8 Time-aligned centre separation in case 3. Pure RL enters the buffer; the filtered methods remain above it in these flights. Distance after a method's arrival is not evaluated because its episode has ended. 20

A.1 Supplementary case 1 separation history for the selected checkpoint. . . 29
A.2 Supplementary case 2 separation history for the selected checkpoint. . . 30

List of Tables

4.1 Principal implementation modules. 11
4.2 PPO and environment settings used in the reported run. 12

5.1 Additional goal tests; twenty goals per distance group. 16
5.2 Main scenario results for the validated checkpoint. $d_{\min}$ is centre separation, not surface clearance. Cases 1–3 are linear, accelerating and figure-eight motion. 17
5.3 Separate comparison using the final 670,000-step checkpoint. PID rows are repeated for consistent comparison and are not independent repeated trials. 21

A.1 Versions recorded in the training manifest. 28
A.2 Figure files and their role. 28

List of Algorithms

1 Hierarchical control at one evaluation sample 10

Abbreviations and Notation

Symbol / abbreviation	Meaning
UAV	Unmanned aerial vehicle
RL / PPO	Reinforcement learning / proximal policy optimization
IMM	Interacting multiple model estimator
CV / CA / CT	Constant-velocity / constant-acceleration / coordinated-turn model
CBF / ACBF	Control barrier function / adaptive control barrier function
PID	Proportional–integral–derivative controller
p, v	UAV position and velocity in the inertial frame
p_g, p_o	Goal position and obstacle centre position
u_{ref}, u	Nominal and filtered net-acceleration commands
$\Delta t, N$	Control interval and prediction horizon
d, d_c, d_s	Centre separation, collision threshold and safety boundary
e	Euclidean position error to the goal
h_i, ψ_i	Predicted barrier value and first-order discrete barrier difference
γ	Adaptive barrier coefficient; distinct from PPO's discount factor
β	PPO reward-discount factor, denoted `ppo_gamma` in code
ξ, δ	Nonnegative higher-order and coefficient-update relaxation variables

Chapter 1

Introduction

1.1 Motivation

Drone navigation in a dynamic environment involves two related requirements: moving toward the destination and preserving separation from obstacles. A controller can satisfy one requirement while failing the other. For example, remaining stationary may avoid an obstacle but fail the mission, while a fast direct trajectory may reach the goal after passing too close to another object. An evaluation should therefore measure arrival, final tracking error, obstacle separation and computational cost together.

Learning-based navigation is attractive because the mapping from state to control can be improved through interaction with a simulator. However, the resulting policy is not automatically a reliable obstacle-avoidance controller. In this project, PPO learns goal-directed motion, while an explicit lower layer uses predicted obstacle motion to modify commands. The design follows the hierarchical idea of Gao and Sun (2026), with implementation adaptations documented throughout this report.

1.2 Problem statement

The simulated system consists of one quadrotor and one moving spherical obstacle. The drone starts from a prescribed position and must enter a goal neighborhood within a finite episode. The nominal policy outputs bounded three-dimensional acceleration commands. The safety layer receives the drone state and an estimate of the obstacle trajectory, and attempts to preserve a prescribed separation while remaining close to the nominal command.

The central research question is: *Can a prediction-based adaptive safety filter improve measured obstacle separation while retaining the goal-reaching behavior of a learned navigation policy?* A second question concerns the cost of that filter in mission time, path length and computation. A third concerns the relationship between successful benchmark trajectories and generalization to other destinations.

1.3 Objectives and scope

The project has four objectives. First, it establishes a consistent interface between PPO, the drone dynamics and the safety filter. Second, it implements obstacle-state estimation and finite-horizon filtering. Third, it compares pure RL, RL with ACBF, and a classical controller with the same ACBF layer. Finally, it evaluates learning progress and investigates failures rather than treating every saved model as converged.

The scope is a simulation study with one drone, one obstacle, prescribed obstacle trajectories and exact simulated obstacle-position measurements. The estimator still uses process and measurement covariance assumptions, but a realistic noisy sensing pipeline is not evaluated. Multi-drone coordination, multiple simultaneous obstacles, wind disturbances and physical flight tests are outside the demonstrated scope.

1.4 Project contribution

The contribution is an implementation and empirical evaluation of a paper-inspired control architecture. It includes a versioned policy interface, consistent plant forecasts, a curriculum-based training procedure, recorded success and safety diagnostics, and checkpoint comparisons that expose residual position bias. It is not a claim of a new PPO algorithm or a new formal barrier-function theorem.

The evidence is deliberately separated into three levels. The training logs show whether the configured validation objective is reached. Additional goal tests examine behavior beyond the callback's fixed goal set. Moving-obstacle tests examine the interaction between navigation and filtering. A strong result at one level is not treated as proof at the other levels.

1.5 Organization of the report

Chapter 2 introduces the relevant learning and control concepts. Chapter 3 formulates the implemented navigation and filtering problem. Chapter 4 describes the software, training and evaluation procedures. Chapter 5 reports the measured results and limitations. Chapter 6 presents conclusions and future work. Appendix A records provenance, reproduction commands and the figure inventory.

Chapter 2

Background and Related Work

2.1 Reinforcement learning and PPO

A policy maps observations to actions, and training adjusts the policy to improve expected return. Let o_t denote an observation, a_t an action and r_t a reward. With discount factor β, the return is $G_t = \sum_{j=0}^{\infty} \beta^j r_{t+j}$. A higher return is useful only insofar as the reward and evaluation criteria reflect the intended task.

PPO limits policy changes using a clipped probability-ratio objective (Schulman et al. 2017). Writing $\rho_t(\theta) = \pi_\theta(a_t \mid o_t)/\pi_{\theta_{\text{old}}}(a_t \mid o_t)$ and $\hat{A}_t$ for an estimated advantage, the policy objective is

$$L^{\text{clip}}(\theta) = \mathbb{E}_t\left[\min\left(\rho_t(\theta)\hat{A}_t, \text{clip}(\rho_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t\right)\right]. \tag{2.1}$$

The present implementation uses Stable-Baselines3 rather than a new implementation of PPO (Raffin et al. 2021). Training samples stochastic actions; the reported policy evaluations use deterministic actions. Consequently, success over a training episode window and success of the final deterministic policy are related but different measurements.

2.2 Safety filtering

A safety filter modifies a nominal control command using constraints derived from the system state. The nominal controller retains responsibility for task progress. Barrier-function methods provide a mathematical language for describing a safe set and restricting control inputs to maintain it under appropriate assumptions (Ames et al. 2017). Those assumptions include the dynamics, available control authority and feasibility of the constraints.

In a numerical implementation, finite sampling, state-estimation error and approximate optimization must also be considered. Therefore a barrier-inspired controller operating successfully in a few simulations is not itself a proof of forward invariance.

This report uses measured geometric separation to assess the local implementation and explicitly records cases where prediction or feasibility limits have appeared.

2.3 Obstacle prediction with multiple models

One motion model may not describe every phase of an obstacle trajectory. A constant-velocity model is suitable for straight motion, a constant-acceleration model represents changes in speed, and a coordinated-turn model represents changing heading. The local IMM predictor combines these alternatives by mixing their state estimates, performing model-specific updates, updating model probabilities and fusing the resulting predictions. This is the predictor structure adopted from the source architecture (Gao and Sun 2026).

The combination is useful for a finite prediction horizon, but its output remains an estimate. A small error matters when a predicted trajectory passes close to the safety boundary. The study therefore distinguishes predicted separation from the separation measured after the simulator executes the command.

2.4 Simulation platform and classical baseline

The `gym-pybullet-drones` environment provides quadrotor simulation and control components suitable for learning experiments (Panerati et al. 2021). This project builds a custom navigation environment around its Crazyflie model. The actual control command is converted to rotor inputs using an inner attitude controller; the vehicle is not advanced solely by an ideal point-mass equation.

A position-error controller provides a classical comparison. It is labelled PID+ACBF to retain the repository terminology, although the evaluated position gains set the integral term to zero. Thus its outer navigation law is effectively proportional–derivative control. All three methods share the same simulated plant and stopping criterion.

2.5 Relationship to the source paper

Gao and Sun (2026) motivates the combination of learned reference commands, IMM prediction and adaptive barrier filtering. The present work evaluates that structural idea, not an exact reproduction of every reported paper experiment. The local policy has expanded observations, net-acceleration semantics, an explicit curriculum and additional reward terms. A single PPO checkpoint is used in both RL variants across all three obstacle cases. These choices make the effect of adding the filter observable

for a fixed policy, but do not reproduce comparisons involving separately trained case-specific policies.

No numerical superiority claim is transferred from the source paper to this implementation. In particular, the local PID baseline remains faster, and the observed safety benefit is reported using explicit centre-distance thresholds. Claims about universal safety, adaptation benefits relative to a fixed CBF, or the independent contribution of IMM require further analysis and experiments.

Chapter 3

System Design and Mathematical Formulation

3.1 Hierarchical architecture

Figure 3.1 shows the implemented information flow. PPO proposes a nominal command from the drone observation. The IMM processes obstacle positions independently of PPO. The ACBF layer combines the predicted obstacle trajectory with candidate drone trajectories to select a bounded command. The inner controller converts that command into rotor inputs. In the pure-RL comparison, the nominal command bypasses the IMM/ACBF layer; the observation and PPO weights are unchanged.

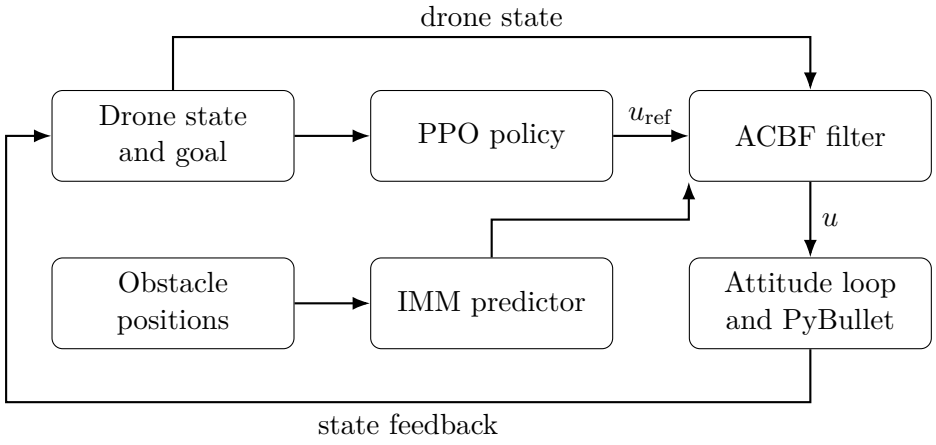


Figure 3.1: Implemented hierarchy. The safety layer is used during evaluation; PPO training uses goal-navigation episodes with a distant obstacle.

3.2 State, observation and action

Let $p, v \in \mathbb{R}^3$ denote inertial position and velocity, and let p_g denote the target. The position error is $p_g - p$. A conceptual translational model is

$$\dot{p} = v, \qquad \dot{v} = -ge_3 + \frac{T}{m} R e_3, \tag{3.1}$$

where R maps body-frame vectors to the inertial frame and T is total rotor thrust. The actual simulator includes rotational dynamics and the inner controller; this equation is not a replacement for the simulated plant.

The normalized policy action $a \in [-1, 1]^3$ is mapped to net acceleration $u = k_{ma}a$, subject to the effective command bounds. Here $k_{ma} = 1$, giving an effective per-axis range of $[-1, 1]$ m/s^2. Although the configuration also contains nominal bounds of $[-3, 3]$, the implementation intersects them with the normalized action range. The desired force is $m(u+ge_3)$, so gravity is added once. A zero net-acceleration command corresponds to a level hover in the ideal stationary case.

The 19-dimensional observation is

$$o = \left[\frac{v}{2},\ \frac{p_g - p}{5},\ \frac{\eta}{\pi},\ \frac{\omega}{10},\ \frac{\eta_{\text{prev}}}{\pi},\ \frac{p}{10},\ f_h\right], \tag{3.2}$$

where η is roll–pitch–yaw, ω is angular velocity, η_{prev} is stored inner-controller attitude history and f_h is the fraction of the required goal-hold interval completed. The fixed scales are part of the checkpoint interface. Obstacle position is not included. Absolute position provides information relevant to flight bounds, but also allows location-dependent policies; its effect is revisited in the evaluation.

3.3 Navigation reward and stopping criterion

Let $e_t = \|p_g - p_t\|$, I_g indicate successful arrival and I_b indicate a flight-boundary failure. The implemented reward is

$$\begin{aligned} r_t = {} & 100I_g + 5(e_{t-1} - e_t) - 10I_b - 0.1\Delta t \\ & - 0.5e_t\Delta t - 0.1\left\|v_t\right\|^2\Delta t. \end{aligned} \tag{3.3}$$

The first four terms follow the terminal/progress/boundary/time structure used in the source approach. The distance and speed costs and the chosen scaling are local adaptations. The speed cost is intended to discourage excessive motion, while the terminal reward distinguishes arrival from merely approaching the destination. These design intentions are assessed through actual success measurements rather than assumed to guarantee convergence.

Arrival requires $e_t < 0.10$ m and $\|v_t\| \leq 0.20$ m/s for five consecutive 20 Hz control samples. The episode limit is 500 samples, or 25 s. Leaving the configured flight bounds or dropping below 0.05 m altitude is a terminal failure. A time limit is a truncation. Geometric obstacle collision is recorded by the evaluator; it is not

conflated with the goal or boundary stopping rule. An episode can therefore reach a goal while still having a safety violation in its recorded trajectory.

3.4 IMM prediction

The implementation maintains native CV, CA and CT states of dimensions 6, 9 and 7, respectively. Mixing is performed through a common nine-dimensional position–velocity–acceleration representation. If μ_i denotes the prior probability of model i and Π_{ij} the transition probability, then

$$\bar{c}_j = \sum_i \Pi_{ij}\mu_i, \qquad \mu_{i|j} = \frac{\Pi_{ij}\mu_i}{\bar{c}_j}. \tag{3.4}$$

Mixed means and covariances initialize the model-specific updates. Measurement likelihoods update the mode probabilities. Each model is then propagated independently over the horizon, and predicted positions are fused using the current probabilities:

$$\hat{p}_{o,k+i|k} = \sum_j \mu_j(k)\hat{p}^{(j)}_{o,k+i|k}. \tag{3.5}$$

The transition matrix has diagonal entries 0.6 and off-diagonal entries 0.2. Process and position-measurement covariance scales are both 0.05. The prediction horizon is $N = 5$, corresponding to 0.25 s. This weighted forecast is not an uncertainty envelope.

3.5 Geometry and adaptive barrier conditions

The geometric centre separation is $d = \|p - p_o\|$. With configured drone and obstacle radii of 0.2 m and a 0.1 m cushion,

$$d_c = 0.4 \text{ m}, \qquad d_s = 0.5 \text{ m}. \tag{3.6}$$

A collision proxy is recorded when $d \le d_c$; a buffer violation is recorded when $d < d_s$. Surface clearance is $d - d_c$. These definitions are kept distinct in every results table.

For predicted positions at horizon index i, define

$$h_i = \tfrac{1}{2}\left(\|\hat{p}_i - \hat{p}_{o,i}\|^2 - d_s^2\right), \tag{3.7}$$

$$\psi_i = h_{i+1} - (1 - \gamma_1)h_i, \tag{3.8}$$

$$b_i = \psi_{i+1} - (1 - \gamma_2\gamma)\psi_i, \tag{3.9}$$

with $\gamma_1 = 0.8$, $\gamma_2 = 0.5$ and $0 \le \gamma \le 1$. The local filter constrains future clearance

values and the higher-order residuals. The adaptive target is based on fractional closing rate. With $p_r = p_o - p$ and $v_r = v_o - v$,

$$q = \max\left(0, -\frac{p_r^{\mathsf{T}} v_r}{\|p_r\|^2}\right), \qquad \gamma^* = \mathrm{clip}(1 - T_q q, 0, 1), \tag{3.10}$$

where $T_q = 2$ s; the implementation protects the denominator near zero. This definition and its numerical realization are implementation choices, not a new safety theorem.

3.6 Numerical safety-filter optimization

The candidate command is held constant over each forecast. Drone positions are obtained from a separate PyBullet world initialized from the live state, rather than only from the ideal acceleration model. The filter minimizes

$$\lambda_u \|u - u_{\mathrm{ref}}\|^2 + \lambda_\gamma(\gamma - \gamma^*)^2 + \lambda_\delta\left(\delta^2 + \|\xi\|^2\right), \tag{3.11}$$

where $\lambda_u = 1$, $\lambda_\gamma = 10$ and $\lambda_\delta = 10^4$. The constraints include command bounds, the coefficient interval, future clearance, and higher-order residuals with nonnegative slack ξ. A coefficient-tracking condition is

$$(\gamma - \gamma^*)^2 \le \delta + (1 - \alpha)(\gamma_{\mathrm{prev}} - \gamma^*_{\mathrm{prev}})^2, \quad \delta \ge 0, \quad \alpha = 0.9. \tag{3.12}$$

Clearance constraints are not given slack in the convex subproblem; the higher-order conditions are relaxed. Finite-difference linearization and a shrinking trust region are used to form convex subproblems. Because the coefficient condition is quadratic, the implementation is more accurately described as a convex quadratic/conic subproblem than as an unconstrained quadratic-program projection.

If the nominal command passes all predicted residual checks, it is returned unchanged. Otherwise CLARABEL is tried first, with SCS as a backup. Finite solutions must pass a numerical feasibility check. Candidate commands are then checked using the nonlinear rollout; up to six linearization passes are allowed. If no acceptable command is found, evaluation stops with a filter-failure status. Stopping the simulated episode is not a demonstrated emergency-flight maneuver.

Algorithm 1: Hierarchical control at one evaluation sample

Input: Drone observation o_k, obstacle position z_k, trained PPO policy
Update IMM using z_k and predict $\hat{p}_{o,k:k+N}$;
Compute deterministic nominal command u_{ref} from PPO;
Predict drone response to the held nominal command;
if *all predicted barrier residuals are nonnegative* **then**
    Return u_{ref};
else
    Linearize residuals and solve the constrained subproblem;
    Verify the candidate using a nonlinear drone rollout;
    Repeat with updated margins if needed, up to the configured pass limit;
    if *a verified command exists* **then**
        Apply the bounded command;
    else
        Record filter failure and terminate evaluation;

3.7 Meaning and limits of safety

Predicted constraint satisfaction is conditional on the forecasts. A measured obstacle can move differently from its predicted position, and an optimization may become infeasible under limited acceleration authority. Higher-order slack further separates the implemented constraint checks from a strict invariance proof. The experiments therefore report observed collision and buffer events, solver diagnostics and execution times. They do not claim that the mathematical guarantees of an idealized barrier construction automatically apply to every simulated or physical trajectory.

Chapter 4

Implementation and Experimental Method

4.1 Software organization

The project uses Python, PyBullet, `gym-pybullet-drones`, Gymnasium, Stable-Baselines3, PyTorch and CVXPY. Table 4.1 identifies the principal components. Experiment manifests record package versions, configuration snapshots, source fingerprints and model hashes. This prevents plots from being silently associated with a different checkpoint.

Table 4.1: Principal implementation modules.

Module	Responsibility
`UAVNavEnv.py`	Reset, observation, reward, dynamics and episode outcomes
`acceleration_ctrl.py`	Net acceleration to attitude and rotor commands
`imm_predictor.py`	Model mixing, estimation and obstacle forecasts
`acbf_filter.py`	Adaptive constraints, optimization and verification
`hierarchical_ctrl.py`	Integration of PPO or PID with IMM and ACBF
`train_ppo.py`	Learning, curriculum, validation and checkpoint selection
`evaluate.py`	Fixed-scenario comparisons and outcome logging
`utils/logger.py`	Trajectories, distances, solver data and figures

The physics and attitude loop run at 240 Hz. PPO and the safety filter issue commands at 20 Hz, so each command spans twelve physics steps. Episode resets clear the attitude-controller history. A shadow simulator copies the drone state and controller memory for forecasts without advancing the live world. The policy interface is checked before evaluation, including action semantics, scales and control frequencies.

4.2 Training configuration

The run analysed in this report began from scratch with seed 42, eight environments and a requested budget of two million aggregate transitions. It did not resume the

previous checkpoint. Its early history matches the earlier run with the same seed and settings; it is therefore not an independent-seed replication.

Table 4.2: PPO and environment settings used in the reported run.

Parameter	Value
Actor and critic hidden layers	$[64, 64]$, Tanh activation
Learning rate	10^{-4}
Steps per environment per rollout	1,024
Environments	8
Aggregate transitions per full rollout	8,192
Minibatch size / epochs	128 / 5
Reward discount β	0.99
PPO clipping / target KL	0.2 / 0.015
Entropy coefficient / initial log standard deviation	0 / -1.5
Control / physics frequency	20 / 240 Hz
Maximum episode length	500 control samples (25 s)
Validation interval	10,000 aggregate transitions

The implementation uses `DummyVecEnv`. Multiple environments supply separate trajectories, but their simulation steps are executed sequentially; this is not eight-way process parallelism. In PPO, rollout size is the product of steps per environment and environment count (Stable-Baselines3 Contributors n.d.). A rollout/training cycle contains many minibatch optimizer steps, so cycle count must not be confused with gradient-update count.

4.3 Curriculum and validation

Goals are initially sampled close to the fixed start $[0, 0, 1]^\mathsf{T}$ m. The curriculum radii are 0.4, 1.0 and 2.5 m, followed by the full goal distribution. At a finite radius r, the sampled distance is between $r/2$ and r, subject to feasible altitude and flight bounds. Promotion requires at least 80% success over a twenty-episode window for the active stage. The full distribution samples horizontal coordinates within approximately $[-5, 5]$ m and altitude within $[0.5, 3]$ m, with initial goal distance greater than 2 m.

Four fixed distant validation goals are

$$\{[-3, 2, 1.8],\ [1, -4, 1.2],\ [2, 2, 2.5],\ [-2, -2, 0.7]\}\ \text{m}. \tag{4.1}$$

Six additional stage-dependent goals are placed along coordinate directions, with a lower-altitude adjustment where needed. These stage goals change when the curricu-

lum advances. Their success curve therefore does not represent an unchanged test set throughout training.

Checkpoint selection ranks the number of successes on the four fixed goals, then their mean final error and mean elapsed time. A separate validated checkpoint is saved when both validation sets pass. Training stops after three successful validations associated with distinct policy updates. The analysed run met this condition at 650,000, 660,000 and 670,000 transitions. A duplicate end-of-training record at 670,000 does not represent an extra independent trial.

4.4 Checkpoint identity and reporting policy

The selected checkpoint at 630,000 transitions is byte-identical to the saved validated model. It is the main model used for scenario figures and tables in this report. The final checkpoint at 670,000 transitions is evaluated separately. Keeping both avoids mixing the most favorable result from one checkpoint with another checkpoint's safety metrics.

The term "validated" refers to the configured finite goal sets, not a certification. All checkpoint identifiers and figure sources are retained in the accompanying artifact manifest. Historical results are used only where they diagnose a failure or illustrate a change, and are labelled as historical.

4.5 Additional goal evaluation

Each checkpoint is evaluated on sixty stored goals: twenty nearby goals at distances 0.2–0.4 m, twenty medium goals at 0.5–1.0 m, and twenty distant goals whose sampled distances range from 2.07 to 6.26 m. The obstacle is placed at $[12, 12, 12]^\mathsf{T}$ m to isolate goal navigation. Actions are deterministic, and the original arrival conditions and 25 s limit are retained.

The goals are the same additional evaluation set used in earlier project reviews. They are separate from the callback's ten validation goals but have been reused for model analysis. Accordingly, they are described as an additional paired evaluation set, not as a newly untouched test set. Each checkpoint receives the exact same goal coordinates and seeds.

4.6 Moving-obstacle scenarios

All scenarios use start $[0, 0, 1]^\mathsf{T}$ m and goal $[5, 0, 2]^\mathsf{T}$ m. The three obstacle laws are

$$p_o^{(1)}(t) = [2.5,\ 0.8t,\ 1.5]^\mathsf{T}, \tag{4.2}$$

$$p_o^{(2)}(t) = [2.5,\ -1.5 + 0.5t + 0.2t^2,\ 1.5]^\mathsf{T}, \tag{4.3}$$

$$p_o^{(3)}(t) = [2.5 + 1.5\sin(0.6t),\ 1.5\sin(0.6t)\cos(0.6t),\ 1.5]^\mathsf{T}. \tag{4.4}$$

The first is linear motion, the second constant acceleration, and the third figure-eight motion. Pure RL and RL+ACBF use the same PPO weights. PID+ACBF uses position gains $K_p = I$, $K_d = 2I$ and $K_i = 0$, with the same prediction and filtering layer. The comparison therefore changes the nominal controller or the presence of the filter, while retaining the plant and measurement protocol.

4.7 Metrics and interpretation

The evaluator records goal success, stop reason, arrival time, terminal position error, terminal speed, sampled minimum centre separation and path length. Path length is the sum of Euclidean distances between consecutive sampled UAV positions. Solver status, relaxation values, forecast error and command-computation time are also recorded.

The safety measurements are sampled at the control frequency and use configured radii; they are not a continuous-time swept-volume proof. Control-computation times are wall-clock measurements, while arrival time is simulation time. Diagnostic evaluations ran concurrently, so timing results are not isolated benchmarks. The distinction is important when assessing whether a controller can meet a 50 ms real-time deadline.

No new learning was performed during the reported checkpoint evaluation. The inference runners explicitly block PPO learning and optimizer updates, and model hashes were verified after testing. This separates measured policy behavior from changes that could otherwise occur during analysis.

Chapter 5

Results and Discussion

5.1 Training progress and early stopping

Figure 5.1 summarizes the recorded validation history. The four distant goals remain fixed, whereas the six stage-validation goals become harder after curriculum promotion. Success on the fixed distant set becomes consistent after approximately 560,000 transitions. Both sets pass at 650,000, 660,000 and 670,000 transitions, triggering the configured early stop. The final policy has 670,000 recorded transitions even though the requested ceiling was two million.

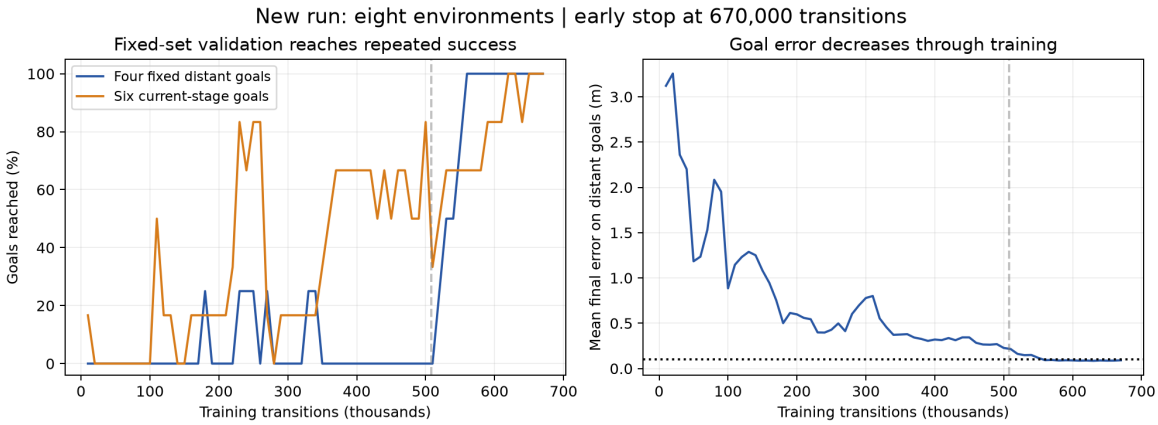


Figure 5.1: Validation history for the reported eight-environment run. The dashed vertical line marks 507,904 transitions, the end of the earlier same-seed run. Current-stage goals change on curriculum promotion. The repeated end-of-training validation is counted only once in this plot.

The run contains 1,639 completed training episodes: 655 goals, 917 timeouts and 67 out-of-bounds terminations. These totals include early unsuccessful learning. In the final hundred episodes, ninety succeed. After the full goal distribution is enabled, 336 of 461 completed episodes succeed and no out-of-bounds termination is recorded. These observations indicate improvement during learning, but do not independently certify the final deterministic policy.

The selected checkpoint at 630,000 transitions passes the four distant goals and all six stage goals. The final model also passes both sets. They differ because selection

uses a lexicographic validation score, not simply the most recent training step. Both are retained and evaluated to make this distinction explicit.

5.2 Additional goal-reaching tests

Table 5.1 and Figure 5.2 report the sixty-goal evaluation. Both checkpoints achieve 59/60 successes, or 98.3% observed success. This is a count on the specified evaluation set, not an estimated universal success probability. The obstacle is distant in these trials, so the measurement concerns navigation without active obstacle avoidance.

Table 5.1: Additional goal tests; twenty goals per distance group.

	Validated 630k		Final 670k	
Group	Success	Mean error (m)	Success	Mean error (m)
Near	20/20	0.0815	20/20	0.0795
Medium	20/20	0.0812	20/20	0.0675
Distant	19/20	0.0834	19/20	0.0851

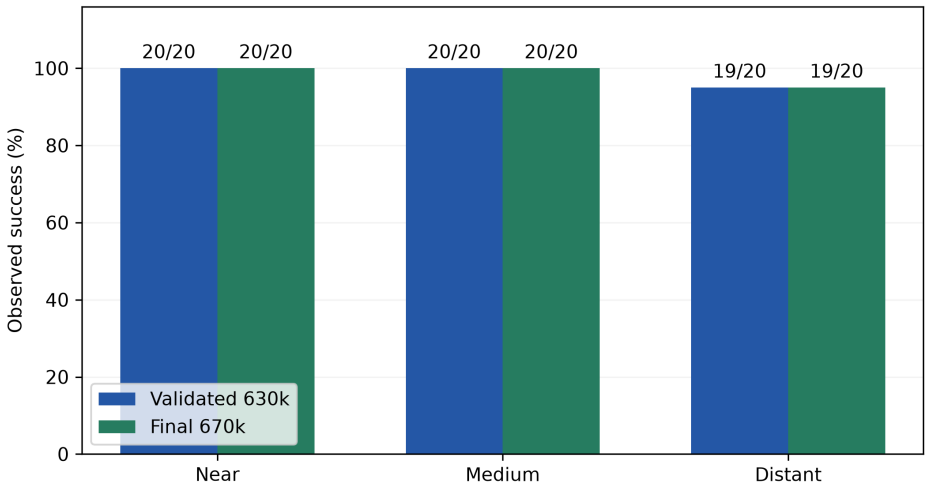


Figure 5.2: Observed success on the same additional goal set for the two checkpoints. No out-of-bounds event occurred in these tests.

Both policies fail the same target, approximately $[0.9412, -4.7667, 2.9977]^\mathsf{T}$ m. The selected checkpoint finishes 0.1097 m away and the final checkpoint 0.1344 m away. Their closest distances are 0.1021 and 0.1034 m, respectively: neither enters the original 0.10 m acceptance radius. Their final speeds are small, indicating a residual position offset rather than a high-speed pass through the target. The tolerance is not relaxed to convert this failure into a success.

The final model averages 6.42 s over the sixty episodes, including the timeout, compared with 7.20 s for the selected model. This aggregate mixes short and long tasks

and should not be interpreted as the time required for the separate five-metre scenario mission. It also shows that the selected and final models are not uniformly ordered on every metric.

5.3 Scenario results for the selected model

Table 5.2 uses only the automatically selected 630,000-step checkpoint. All nine controller/scenario combinations reach the shared goal. The nominal PPO is identical between pure RL and RL+ACBF. Figure 5.3 summarizes arrival and separation, and Figure 5.4 shows the goal-error histories.

Table 5.2: Main scenario results for the validated checkpoint. $d_{\min}$ is centre separation, not surface clearance. Cases 1–3 are linear, accelerating and figure-eight motion.

Case	Controller	Time (s)	Error (m)	$d_{\min}$ (m)	Path (m)
1	Pure RL	13.70	0.089	2.067	7.628
1	RL+ACBF	13.70	0.089	2.067	7.628
1	PID+ACBF	7.60	0.086	1.828	5.087
2	Pure RL	13.70	0.089	1.072	7.628
2	RL+ACBF	13.80	0.088	1.211	7.386
2	PID+ACBF	7.85	0.087	1.003	5.095
3	Pure RL	13.70	0.089	0.464	7.628
3	RL+ACBF	13.60	0.078	0.770	7.441
3	PID+ACBF	7.80	0.085	0.842	5.099

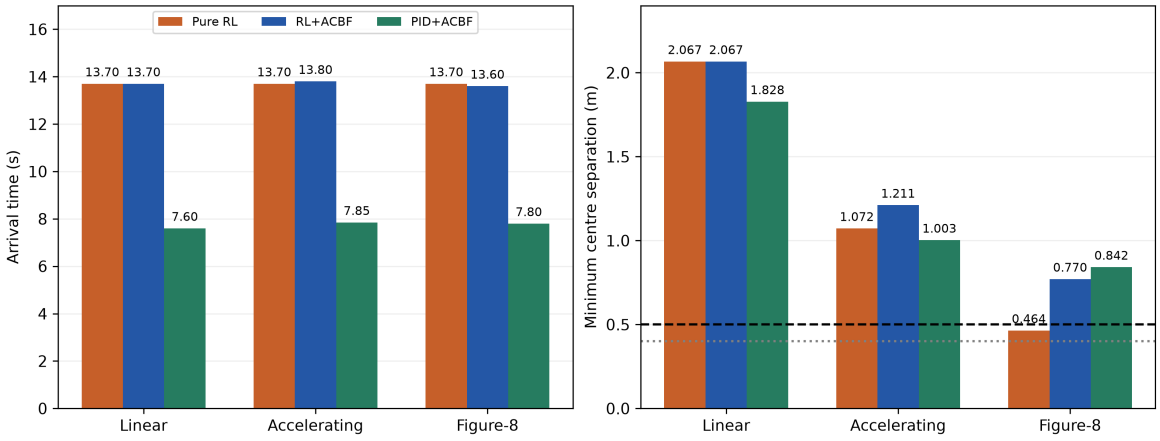


Figure 5.3: Selected-checkpoint comparison. In the right panel the dashed line is the 0.50 m safety boundary and the dotted line is the 0.40 m collision threshold.

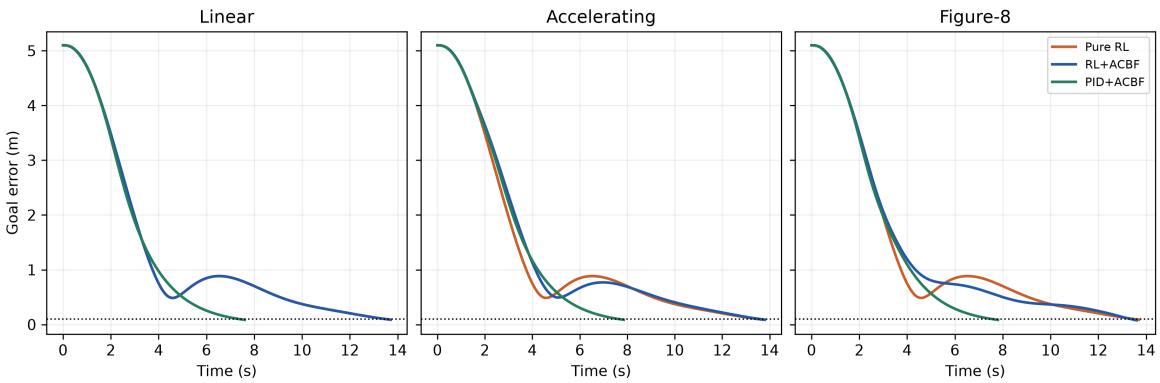


Figure 5.4: Goal-error histories for the selected checkpoint. The dotted horizontal line is the 0.10 m position tolerance; arrival additionally requires the speed and hold-time conditions.

5.3.1 Linear obstacle

In case 1, pure RL and RL+ACBF have the same trajectory and arrive in 13.70 s. Their minimum centre distance is 2.067 m, comfortably above the buffer boundary. The nominal trajectory does not require alteration by the filter in this case. PID+ACBF reaches the goal in 7.60 s with a shorter path. Thus the filter's pass-through behavior can be observed without implying an efficiency advantage for RL.

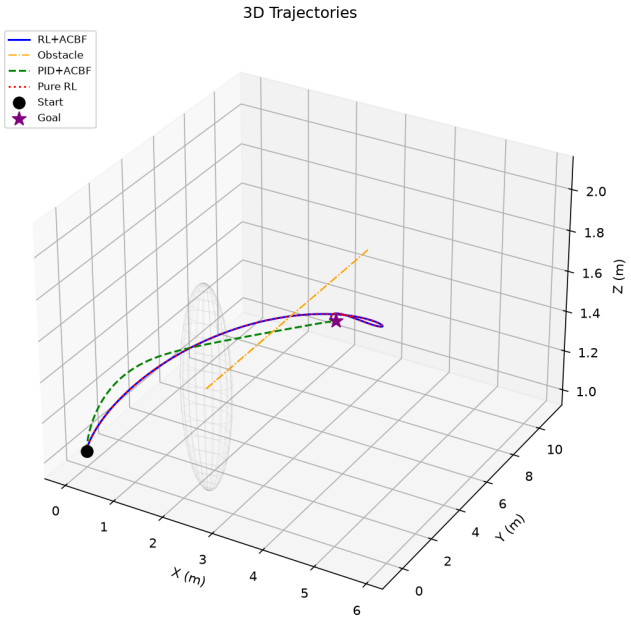


Figure 5.5: Case 1 trajectories for the selected checkpoint. A spatial trajectory plot does not by itself establish time-aligned obstacle clearance; that is measured from the logged states.

5.3.2 Accelerating obstacle

In case 2, the obstacle accelerates across the nominal flight region. Adding ACBF increases minimum centre separation from 1.072 to 1.211 m. Arrival changes from 13.70 to 13.80 s, while sampled path length decreases from 7.628 to 7.386 m. A filter-induced change in state can affect the later policy trajectory, so a larger safety distance does not necessarily imply a longer path. These are measured outcomes for this case, not a general optimality statement.

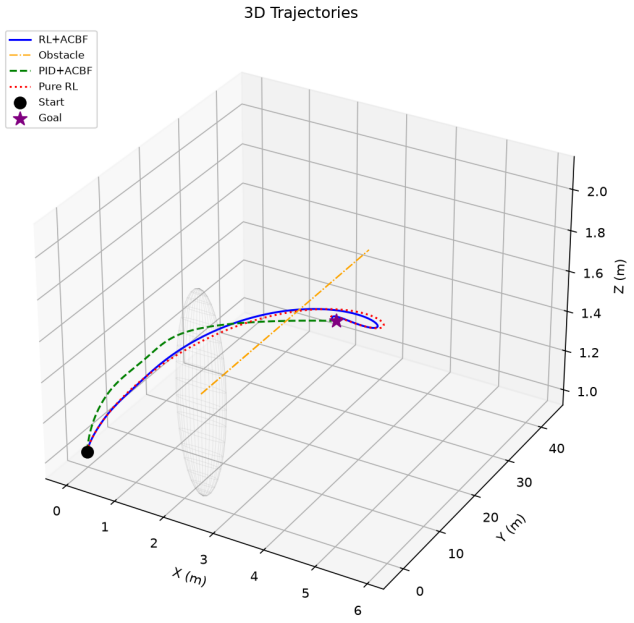


Figure 5.6: Case 2 trajectories for the selected checkpoint. Both RL variants arrive, while the filtered path maintains a larger sampled minimum separation.

5.3.3 Figure-eight obstacle

Case 3 is the clearest demonstration of the filter's measured safety-buffer benefit. Pure RL reaches the goal but attains a minimum centre distance of 0.464 m. This is below the 0.50 m safety boundary and above the 0.40 m geometric collision threshold. It is a buffer violation, not a recorded collision.

RL+ACBF maintains a minimum centre distance of 0.770 m and reaches the goal in 13.60 s. The increase in minimum separation is approximately 0.306 m. Surface clearance changes from 0.064 to 0.370 m. The filtered trajectory also arrives 0.10 s earlier than pure RL. This small time difference should not be interpreted as a reliable speed advantage across tasks, but it shows that preserving the observed buffer need not prevent arrival.

PID+ACBF reaches the goal in 7.80 s and maintains 0.842 m minimum separation.

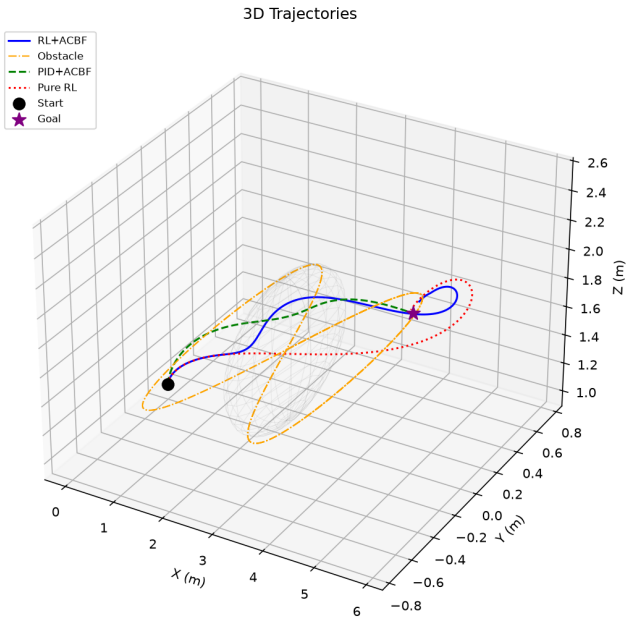


Figure 5.7: Case 3 trajectories for the selected checkpoint. PPO+ACBF and PID+ACBF use the same obstacle predictor and filter but different nominal commands.

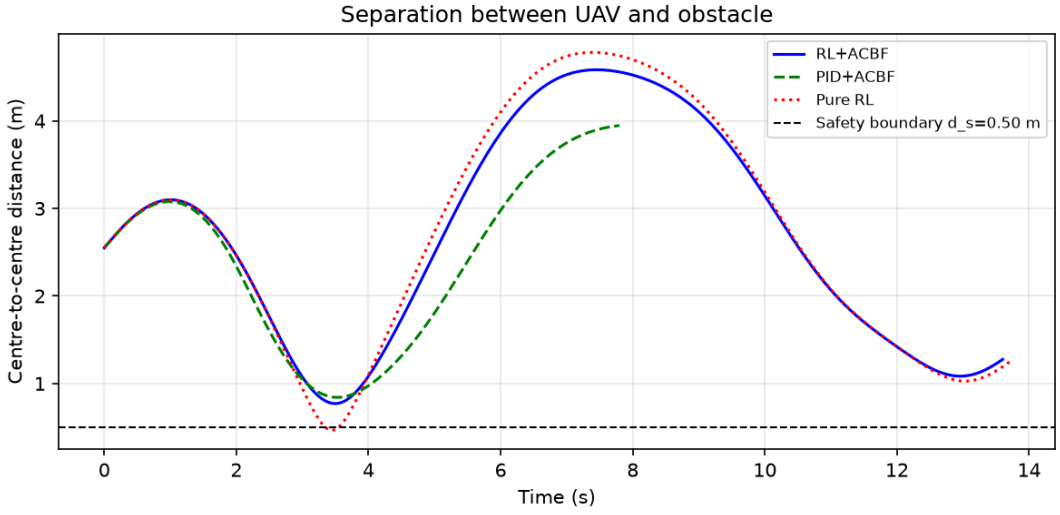


Figure 5.8: Time-aligned centre separation in case 3. Pure RL enters the buffer; the filtered methods remain above it in these flights. Distance after a method's arrival is not evaluated because its episode has ended.

It therefore remains the stronger controller on both arrival time and minimum separation in this particular case. More generally, its mean scenario arrival time is 7.75 s, compared with 13.70 s for selected PPO+ACBF. The local results do not support a claim that RL is superior to PID on all evaluated metrics.

5.4 Final-checkpoint comparison

Table 5.3 reports the final 670,000-step checkpoint separately. It reaches the shared goal in every case, but its behavior differs from the selected checkpoint. Pure RL takes 17.10 s in each case and enters the figure-eight buffer. ACBF preserves the buffer and leads to earlier arrival in cases 2 and 3, at 12.15 and 12.70 s. The safety filter changes the entire subsequent closed-loop trajectory, so it can sometimes improve arrival time even though its objective is to stay close to the nominal command at each sample.

Table 5.3: Separate comparison using the final 670,000-step checkpoint. PID rows are repeated for consistent comparison and are not independent repeated trials.

Case	Controller	Time (s)	Error (m)	$d_{\min}$ (m)	Path (m)
1	Pure RL	17.10	0.097	2.042	7.198
1	RL+ACBF	17.10	0.097	2.042	7.198
1	PID+ACBF	7.60	0.086	1.828	5.087
2	Pure RL	17.10	0.097	1.040	7.198
2	RL+ACBF	12.15	0.096	1.186	6.799
2	PID+ACBF	7.85	0.087	1.003	5.095
3	Pure RL	17.10	0.097	0.465	7.198
3	RL+ACBF	12.70	0.085	0.749	6.853
3	PID+ACBF	7.80	0.085	0.842	5.099

The final and selected models tie on additional goal success but differ in arrival times and the residual error of their shared failed goal. Retaining both provides a more complete picture than reporting whichever checkpoint is best for each metric. The results in the main trajectory figures all use the selected checkpoint.

5.5 Diagnosis of earlier near-goal failures

An earlier run's selected checkpoint at 340,000 transitions approached the scenario goal but stopped outside the tolerance. Pure RL finished approximately 0.424 m away after 25 s. Extending an obstacle-distant diagnostic flight to 60 s left it almost stationary 0.407 m away. A static policy probe found a zero-command position match-

ing that endpoint closely. This identified a learned position bias, rather than merely insufficient episode duration.

Translated short-distance tests provided further evidence: the same 0.2 m relative movement succeeded near the fixed training start but failed near the distant scenario target. Since the observation contains absolute position, a policy can learn different local responses at different locations. The curriculum's early concentration around a fixed start is a plausible contributor. However, no controlled retraining ablation has established that removing absolute position alone would fix this behavior. The latest model substantially reduces the problem, but the remaining distant-goal timeout shows that precise positioning is not perfect everywhere.

5.6 Computational and safety limitations

For the selected PPO+ACBF checkpoint, recorded 95th-percentile control times are approximately 23.4, 116.2 and 118.8 ms in cases 1–3. The intended control interval is 50 ms. These timing measurements were collected during concurrent diagnostic work, so an isolated benchmark is required before drawing a hardware-speed conclusion. Nevertheless, the current measurements do not demonstrate real-time feasibility. Simulation arrival times must not be presented as wall-clock execution times.

The selected PPO+ACBF runs include one accepted inaccurate-solver step in case 2 and three in case 3. The implementation checks finite values and numerical residuals before accepting commands, but also permits higher-order slack. Successful geometry in these flights is an empirical result, not a proof that all barrier conditions hold exactly under every forecast error.

A previous checkpoint exposed a more serious limitation: in a figure-eight encounter, obstacle prediction overestimated next-step separation, the measured distance entered the safety buffer, and the following optimization was infeasible. The current policy avoids that failure in the tested trajectories, but no change to the predictor or filter has eliminated the underlying risk. Better navigation can reduce difficult encounters without making the safety layer universally robust.

5.7 Threats to validity

The results use a single training seed. The two reported checkpoints come from one run and must not be counted as independent learning repetitions. The additional goals are reused evaluation goals. The moving-obstacle cases share a start and destination,

so three successful cases do not constitute three distinct destination-generalization tests. The environment uses prescribed obstacle motion and exact simulated position measurements, not a hardware perception stack.

There is no fixed-CBF baseline, no predictor ablation and no multiple-obstacle test. Therefore the study can attribute the paired difference to adding the complete IMM/ACBF layer for the same PPO policy, but cannot isolate how much of that difference comes from adaptation, the predictor, or other implementation choices. Collision counts are geometric, sampled metrics, and zero observed collisions in a finite set is not a universal probability statement.

5.8 Discussion

The evidence supports three conclusions. First, the implemented training procedure produces substantially improved navigation and satisfies its configured stopping condition. Second, the lower layer prevents the observed figure-eight buffer entry while retaining goal arrival for the evaluated policies. Third, the classical baseline remains faster and more direct. Together these results demonstrate a working hierarchical comparison with useful diagnostics, while defining clear boundaries on what can be claimed.

The most productive next step is broader validation rather than automatically increasing the training budget to its original ceiling. Independent seeds, varied start positions, new goal sets, prediction-error tests and isolated timing measurements would address the most consequential remaining uncertainties.

Chapter 6

Conclusions and Future Work

6.1 Conclusions

This project implemented a hierarchical UAV navigation controller that combines PPO reference commands, IMM obstacle prediction and an adaptive barrier-based safety filter. The implementation was evaluated against pure PPO and a classical PID-based controller using the same simulated plant and arrival conditions. Configuration snapshots, checkpoint hashes and per-step logs were used to make the comparisons traceable.

The latest run stopped at 670,000 transitions after repeatedly passing its fixed validation criteria. The selected checkpoint at 630,000 transitions and the final checkpoint each reached 59 of 60 additional goals. One distant target retained a small steady position error beyond the 0.10 m radius. Thus the configured validation objective was achieved, but arbitrary-goal reliability was not established.

All three controllers reached the common destination in the linear, accelerating and figure-eight obstacle scenarios. For the selected checkpoint, the figure-eight pure-RL path entered the safety buffer with a minimum centre separation of 0.464 m. PPO+ACBF maintained 0.770 m and reached the goal in 13.60 s. This is a concrete observed benefit of the complete filtering layer. It is not a claim that a physical collision was prevented, because the unfiltered trajectory did not cross the configured collision threshold.

The PID-based baseline remained more efficient, reaching the scenario goal in approximately 7.6–7.85 s. The contribution is therefore a successful implementation, comparison and diagnosis of a hierarchical learning-and-filtering approach, rather than demonstrated superiority over classical control. Earlier failures and current timing limitations show why numerical prediction-based safety must be assessed beyond a favorable trajectory plot.

6.2 Future work

Navigation generalization. Repeat training with independent seeds and evaluate on new goals and starting states. Maintain close-to-goal tasks at varied locations after curriculum promotion to test and reduce location-dependent position bias. Use separate model-selection and final-evaluation sets.

Prediction robustness and feasibility. Introduce controlled sensor noise, obstacle acceleration changes and model mismatch. Evaluate margins that account for prediction uncertainty. Develop and test a feasible backup control strategy, since terminating an evaluation on solver failure is not an emergency behavior for a real drone.

Component ablations. Compare adaptive and fixed barrier coefficients, single-model and IMM predictions, and alternative nominal controllers under the same conditions. This would distinguish the contribution of individual components from the effect of the complete hierarchy.

Computational validation. Measure timing in isolated runs and profile the repeated simulator forecasts and optimization. Demonstrate deadline compliance before considering hardware deployment. Hardware-in-the-loop and physical tests should follow only after the simulation assumptions and recovery behavior are made explicit.

Broader scenarios. Extend the setting to multiple obstacles, varying goal locations and disturbances. Report both mission completion and safety events, retaining failures rather than relaxing criteria after observing results. These steps would turn the current successful demonstrations into stronger evidence of repeatable navigation performance.

References

Ames, A. D., X. Xu, J. W. Grizzle, and P. Tabuada (2017). "Control Barrier Function Based Quadratic Programs for Safety Critical Systems". In: *IEEE Transactions on Automatic Control* 62.8, pp. 3861–3876. DOI: 10.1109/TAC.2016.2638961.

Gao, J. and J. Sun (2026). "Hierarchical Safe Reinforcement Learning Control for UAVs via Adaptive Control Barrier Function and Interacting Multiple Model". In: *Advanced Mechatronics* 1.1, 3. DOI: 10.53941/am.2026.100003. URL: https://www.sciltp.com/journals/am/articles/2607004775.

Panerati, J., H. Zheng, S. Zhou, J. Xu, A. Prorok, and A. P. Schoellig (2021). *Learning to Fly—a Gym Environment with PyBullet Physics for Reinforcement Learning of Multi-agent Quadcopter Control*. arXiv: 2103.02142. URL: https://arxiv.org/abs/2103.02142.

Raffin, A., A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann (2021). "Stable-Baselines3: Reliable Reinforcement Learning Implementations". In: *Journal of Machine Learning Research* 22.268, pp. 1–8. URL: https://jmlr.org/papers/v22/20-1364.html.

Sarkar, T. (2026). *BTP UAV Navigation: Local Implementation and Experiment Artifacts*. Local project archive. Training and evaluation run identifiers, checkpoint identities and reproduction details are listed in Appendix A.

Schulman, J., F. Wolski, P. Dhariwal, A. Radford, and O. Klimov (2017). *Proximal Policy Optimization Algorithms*. arXiv: 1707.06347. URL: https://arxiv.org/abs/1707.06347.

Stable-Baselines3 Contributors (n.d.). *PPO Documentation*. URL: https://stable-baselines3.readthedocs.io/en/master/modules/ppo.html (visited on 11/05/2026).

Chapter A

Reproducibility and Artifacts

A.1 Experiment identity

The main results come from local project artifacts (Sarkar 2026). Training run: `20261105T022305_489044Z`. Evaluation root: `20261105T081629_168420Z`. The experiment source fingerprint, using ordered relative Python paths and LF-normalized file bytes, is

`e27217395c51bdde1b11697346c61b5e5ea5a16e159b46739ebe0317d3d758f1.`

The selected `best_candidate.zip` and `validated_model.zip` are identical. Their SHA-256 is

`0b58672c4e2d794b19e04e02ef9c4c56ab091c087481f23aa9d7b45f1d23bd1a.`

The final model SHA-256 is

`ae7a2a0049fb52dbafc69a7d9918496442694bd0fd4d1cce843fd426c12a590c.`

Hashes were verified unchanged after inference. The selected model has 630,000 recorded transitions and the final model 670,000. Both are in the full-distribution curriculum stage.

A.2 Recorded dependencies

These are recorded environment versions, not a claim that every later library version reproduces identical trajectories. The report does not infer hardware speed from these package versions.

A.3 Evaluation commands

The following command evaluates the selected checkpoint and creates a new dated output directory. It does not train a policy.

Table A.1: Versions recorded in the training manifest.

Component	Version
Python	3.14.3
NumPy / SciPy	2.5.3 / 1.18.0
PyTorch	2.14.1+cu126
Gymnasium	1.3.0
Stable-Baselines3	2.9.0
PyBullet	3.2.7
gym-pybullet-drones	2.2.0
CVXPY	1.9.3
CLARABEL / SCS	0.11.1 / 3.3.1
Matplotlib	3.11.2

```
python evaluate.py --model results/training/20261105T022305_489044Z/
    validated_model.zip --case all
```

Listing A.1: Evaluate the selected policy on all three scenarios.

For the final-policy comparison, replace the filename by `ppo_final.zip`. The saved evaluation runner in the review directory performs the sixty-goal tests and blocks learning calls. It uses the stored coordinates and seeds to keep the checkpoint comparison paired. A rerun should use a new output directory rather than overwrite prior evidence.

A.4 Figure inventory

The main external figure files are listed below. Every trajectory and separation figure uses the selected 630,000-step checkpoint. The training figure covers the whole run; the additional-goal figure explicitly compares both checkpoints. A machine-readable `figure_sources.json` is included in the Overleaf package.

Table A.2: Figure files and their role.

File in `figures/`	Purpose
`learning_progress.png`	Fixed and stage validation history
`goal_generalization.png`	Additional goal success for 630k and 670k
`selected_comparison.png`	Main arrival and separation comparison

File	Purpose
`selected_goal_errors.png`	Goal-error curves for the selected policy
`case1_trajectory.png`	Linear-obstacle 3D trajectories
`case2_trajectory.png`	Accelerating-obstacle 3D trajectories
`case3_trajectory.png`	Figure-eight 3D trajectories
`case3_separation.png`	Figure-eight time-aligned safety evidence
`case1_separation.png`	Supplementary linear-case distance history
`case2_separation.png`	Supplementary accelerating-case distance history

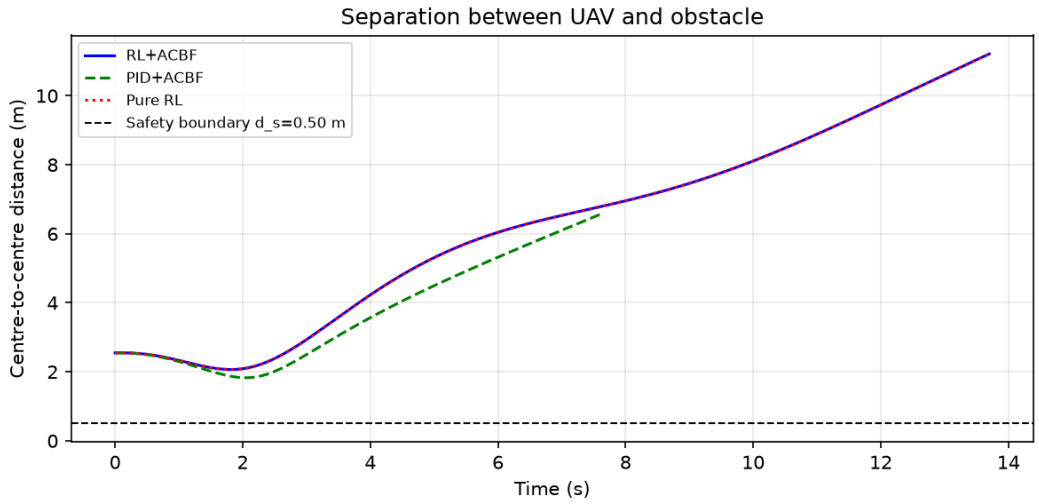


Figure A.1: Supplementary case 1 separation history for the selected checkpoint.

A.5 Interpretation checklist

An arrival claim must use both position and speed/hold conditions. A safety claim must distinguish the 0.50 m buffer from the 0.40 m collision threshold. A checkpoint comparison must identify the actual weights used. A generalization claim must identify which goals and training seeds were tested. A real-time claim requires wall-clock deadline measurements. These distinctions are necessary to reproduce the conclusions of this report without overstating the evidence.

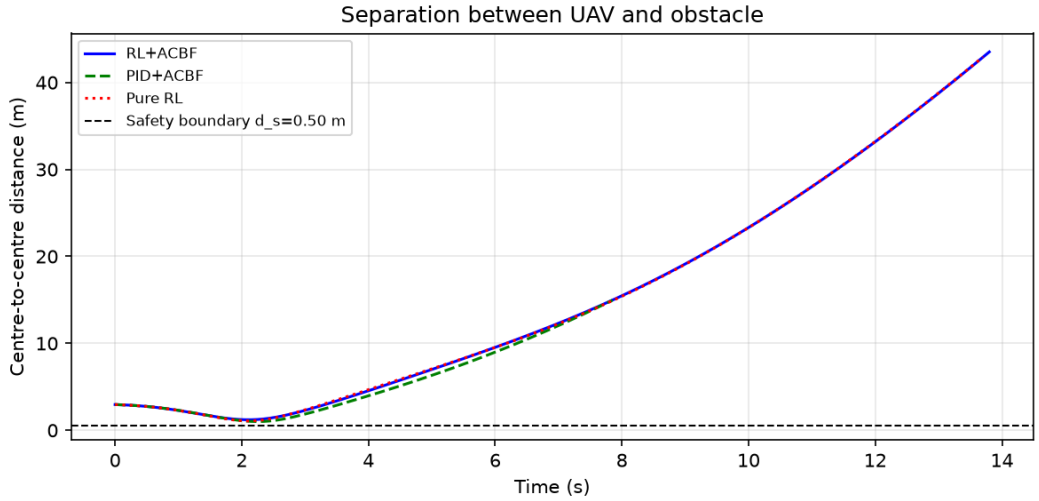


Figure A.2: Supplementary case 2 separation history for the selected checkpoint.